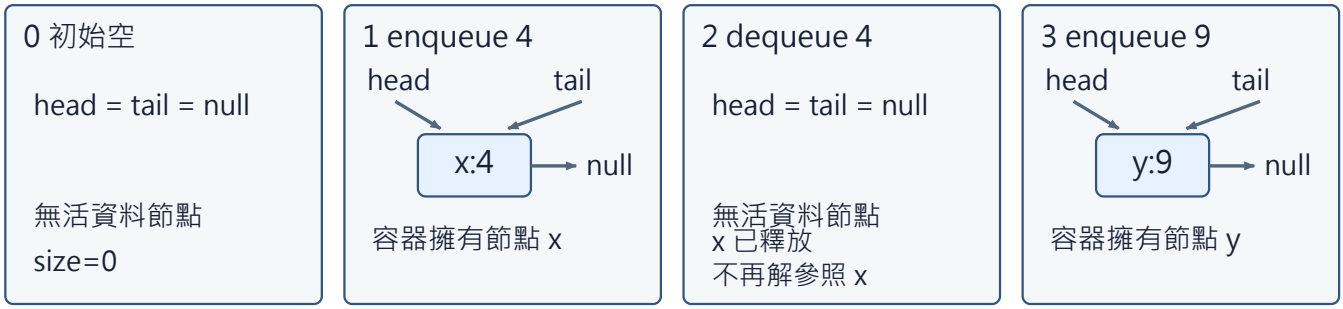
第 4 章推導解答

DS04-E01 • 最後一個佇列節點

初始 head=tail=null。enqueue 4 建新節點 x，令 head=tail=x、x.next=null。dequeue 保存值並移除 x 後，head=null；因佇列已空，tail 也必須設 null，再釋放 x。enqueue 9 建 y，令 head=tail=y。若第二步 tail 仍指 x，下一次沿 tail.next 寫入會使用已釋放物件。這是所有權和生命週期違規，不只是顯示內容不正確。

移除最後一點：先清 head 與 tail，再釋放該節點



DS04-E02 • 原地反轉證明

初始 prev 空、curr=3→8→11。第一輪 prev=3、curr=8→11；第二輪 prev=8→3、curr=11；第三輪 prev=11→8→3、curr 空。每輪先保存 curr.next，才反轉當前 link，故未處理後綴仍可達。兩段無環且節點互斥，聯集固定為原節點集合。終止後 head=prev，舊 head 即新 tail。空串列時兩者仍為 null，單點時 head 與 tail 指同一點。

DS04-E03 • 多項式正規化與相加

P 有同指數 4 重複且未嚴格遞減，合併後 $P=x^4+x$。$Q=-x^4+5$，相加得到 $x+5$，四次項係數為 0 因而不保留節點。任意順序的 p 個輸入項若採比較排序，正規化為 O(p log p) 排序加 O(p) 合併。兩個已正規化串列相加才是 O(p+q)，不能把未排序原始輸入的成本一併忽略。

DS04-E04 • 雙向串列刪除

先令 p=x.prev、q=x.next，再設定 p.next=q、q.prev=p，最後釋放 x 並減少 size。其餘 links 沒變，而改動的兩鄰點滿足 p.next.prev=p、q.prev.next=q，因此一致性恢復。前提為 x 是本容器活著的資料節點，非 sentinel，且沒有重複刪除。若外部可交入任意 pointer，邊界需有 owner 驗證或不接受該介面，不能假設任意指標都合法。

本段固定採兩個不含資料的 sentinel H、T。H.prev=null、T.next=null；空串列為 H.next=T、T.prev=H，size=0。每個資料節點的 prev、next 均非 null，因此以下 p.next 與 q.prev 可安全解參照。若改採無 sentinel 的 null 端點，刪首／尾節點必須另更新 head／tail，不能直接套此程式。

只有一筆資料時 H↔x↔T，刪除取 p=H、q=T，接回 H.next=T、T.prev=H，size 由 1 降 0；H、T 保留且絕不釋放。刪多筆串列的第一或最後資料同樣由鄰接 sentinel 接回。合法節點 handle 與所屬容器的確認在進入指標操作前完成。

DS04-E05

入列 A 得 `A→A`、tail=A；入列 B 得 `A→B→A`、tail=B；出列 A 後為 `B→B`、tail=B；再出列 B 時先將 tail 設為 null，再釋放 B，佇列為空；入列 C 得 `C→C`、tail=C。若釋放 B 後仍保留 tail=B，下一次讀取 `tail.next` 會解參照已釋放物件。

DS04-E06

$\mu = 2$、$\lambda = 3$。第一階段 `(slow,fast)` 為 `(1,2)`、`(2,4)`、`(3,3)`；一個指標重設至 0 後依序為 `(0,3)`、`(1,4)`、`(2,2)`，入口為節點 2。比較節點身分或有效指標，而不是比較儲存值。

DS04-E07

空鏈 fast 為 null，第一個條件即失敗；`A→null` 的 fast.next 為 null，不進迴圈；`A→A` 的 fast 與 fast.next 都非 null，第一輪兩指標均回到 A，判定有環。短路檢查確保只有 fast 非 null 才讀 fast.next，且 fast.next 非 null 時才繼續讀其 next。

DS04-E08

第一步後 p.next=x；第二步讀取的 p.next 已是 x，故 x.next=x，從 head 到達 `3→5→5→⋯`。原節點 8、11 失去由容器 head 出發的擁有路徑，即使仍有外部位址也不能當作容器正確保有它們。正確做法先保存 `old=p.next`，令 `x.next=old`，最後提交 `p.next=x`，得到 `3→5→8→11→null`。

DS04-E09

$\mu = 0$、$\lambda = 2$。第一輪 slow=B、fast=A，兩節點值雖皆為 7，身分不同；第二輪 slow=fast=A，第一次相遇即為入口。把另一指標重設到 head=A 時兩者已相等，不需再移動。若只比較值，第一輪就會把 B 與 A 誤判為同一節點。

DS04-E10

先保存 `p=x.prev=H`、`q=x.next=T`，再設 `p.next=q` 與 `q.prev=p`，最後釋放 x 並將 size 由 1 減到 0。結果 `H.next=T`、`T.prev=H`，兩個 sentinel 仍在；x 的外部指標在釋放後失效，不可解參照。

DS04-E11

淺複製使兩個 head 指向同一鏈，若兩個析構流程都釋放它，後者會重複釋放或存取已釋放節點，造成未定義行為。深複製要建立獨立節點鏈並明訂失敗處理；不可複製契約則刪除複製建構與複製指定，並另行設計 move。tail 雖不擁有節點，仍只能指向活的最後節點；刪除尾節點或清空時必須更新，否則成為懸空指標。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；C++ 標準草案 forward_list、list 與 unique_ptr。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study